STUDENT NAME: AMANDEEP	MODULE: My Practice	PRACTICE SET 002
BATCH: ABES_JAS26_FSD JAVA_IIIYear_Batch 02	DOMAIN: Gen AI & Prompt Engineering	STATUS: In Progress

Practice Set 2 – Prompt Clarity and Rewriting

Learn to diagnose prompt ambiguity, enforce strict document synthesis guidelines, construct rigorous code debugging prompts, and formulate objective technology comparison matrices.

KEY LEARNING OBJECTIVES:

- Disambiguate vague, open-ended user requests into precise technical directives.
- Design executive document summarization prompts with risk matrices and zero fluff.
- Engineer root-cause debugging prompts that isolate exceptions, concurrency issues, and reproduction steps.
- Formulate weighted multi-attribute technology comparison frameworks.

STEP-BY-STEP WORKSHEET INSTRUCTIONS:

Step 1: Identify ambiguous terms, unstated assumptions, and missing boundary conditions in the baseline prompt.
Step 2: Inject concrete technical parameters, environment details, and input/output contracts.
Step 3: Mandate structured output schemas (e.g., Markdown comparison tables, RFC-style summaries, or unit test templates).
Step 4: Validate that your rewritten prompt leaves zero room for speculative or hallucinated answers.

EXERCISE 2.1: DIAGNOSING & ELIMINATING AMBIGUOUS TECHNICAL PROMPTS

Problem Statement: A junior developer submits the vague prompt: 'Tell me how to improve performance.' Redesign this prompt into an exhaustive performance optimization directive that isolates backend database query indexing, JVM garbage collection tuning, and frontend bundle tree-shaking, requiring concrete diagnostic commands (e.g., EXPLAIN ANALYZE, GC log flags, Lighthouse metrics).

Baseline / Reference Prompt:
"Tell me how to improve performance in my application."

STUDENT RESPONSE WORKSPACE:
[1] Role & Persona: _____
[2] Task & Directives: _____
[3] Constraints & Format: _____
[4] Complete Engineered Prompt:

EXERCISE 2.2: STRUCTURED EXECUTIVE DOCUMENT SUMMARIZATION

Problem Statement: Craft a prompt that digests a dense 15-page enterprise technical RFP. The prompt must extract: (1) Executive Summary (strictly 75-100 words), (2) Top 5 Architectural Risk Factors formatted in a high-priority table, (3) Deliverable Milestone Timeline, and (4) Cost/Resource Allocation breakdown, prohibiting corporate jargon.

Baseline / Reference Prompt:
"Summarize this technical software paper into bullet points with risks."

STUDENT RESPONSE WORKSPACE:
[1] Role & Persona: _____
[2] Task & Directives: _____
[3] Constraints & Format: _____
[4] Complete Engineered Prompt:

EXERCISE 2.3: SYSTEMATIC MULTI-THREADED CODE DEBUGGING & STACK TRACE TRIAGE

Problem Statement: Engineer a prompt for an automated senior debugging assistant to analyze a Java ConcurrentModificationException occurring in an asynchronous inventory service. The prompt must require: Root Cause Analysis, Minimal Reproducible Example, Two Alternative Thread-Safe Refactorings (e.g., CopyOnWriteArrayList vs Synchronized Stream), and JUnit 5 concurrency test verification.

Baseline / Reference Prompt:
"Fix this Java error: ConcurrentModificationException in ArrayList during foreach loop."

STUDENT RESPONSE WORKSPACE:
[1] Role & Persona: _____
[2] Task & Directives: _____
[3] Constraints & Format: _____
[4] Complete Engineered Prompt:

EXERCISE 2.4: MULTI-ATTRIBUTE TECHNOLOGY COMPARATIVE EVALUATION

Problem Statement: Construct an objective comparative prompt evaluating PostgreSQL vs MongoDB for a high-volume IoT manufacturing telemetry platform. The prompt must mandate a 6-dimension evaluation matrix (Throughput, ACID Consistency, Schema Evolution, Query Complexity, Storage Cost, and Operational Overhead) with quantitative 1-10 scoring and an architectural recommendation.

Baseline / Reference Prompt:
"Compare PostgreSQL and MongoDB for a big data IoT project and tell me which is better."

STUDENT RESPONSE WORKSPACE:
[1] Role & Persona: _____
[2] Task & Directives: _____
[3] Constraints & Format: _____
[4] Complete Engineered Prompt:

SELF-REVIEW CHECKLIST:

- [ ] **Disambiguation:** All vague terms replaced with concrete technical scope and benchmarks.
- [ ] **Environment Context:** Runtime, tech stack, and versioning parameters explicitly specified.
- [ ] **Structured Deliverables:** Explicit tables, code blocks, or delimited sections enforced.
- [ ] **Actionable Verification:** Requires verification tests, profiling commands, or validation steps.
- [ ] **Zero Hallucination:** Prompts require justification from provided data or formal documentation.

CRITICAL REFLECTION & LEARNING SYNTHESIS:

- *What specific ambiguities in the baseline prompt caused the model to produce generic or unhelpful advice?*
- *How does demanding structured comparison tables reduce confirmation bias when evaluating competing technologies?*

Reflection Notes:

_____
_____

COMPLETION STATUS:

[ ] COMPLETED (Meets All Rubric Criteria)
[ ] NEEDS REVISION (Resubmit Refined Prompts)

Evaluator Signature: _____

EVALUATOR & MENTOR REMARKS:

Feedback Notes:
_____
Rubric Evaluation: [ ] Clarity [ ] Constraints [ ] Edge Handling